Архитектура игры для проекта

Ученик: **Лучкин М.П.**
Группа: **10.3**

13 марта 2026 г.

Содержание

1 Общая структура программы **2**

2 Описание ключевых интерфейсов и абстрактных классов **2**
2.1 Интерфейс ICommand 2
2.2 Абстрактный класс CommandBase 2
2.3 Интерфейс ICondition 3
2.4 Абстрактный класс ConditionBase 3
2.5 Интерфейс IEffect 3
2.6 Абстрактный класс EffectBase 3
2.7 Абстрактный класс GameEventBase 3
2.8 Интерфейс IInteractable 4

3 Конкретные реализации **4**
3.1 Команды 4
3.2 Условия 4
3.3 Эффекты 5
3.4 События 5
3.5 Объекты взаимодействия 5

4 Композиция объектов на примере локации **5**

5 Класс Game и игровой цикл **6**

6 Соответствие требованиям ТЗ **6**

1 Общая структура программы

Программа состоит из следующих ключевых компонентов:

- **Game** — главный класс, управляющий игровым циклом и состоянием.
- **GameState** — хранилище текущего состояния мира (здоровье, инвентарь, флаги, счётчик ходов, журнал).
- **Location** — локация, содержащая объекты, события и переходы.
- **Player** — модель игрока (может быть частью GameState).
- **Команды** — реализуют интерфейс `ICommand`.
- **Условия** — реализуют интерфейс `ICondition`.
- **Эффекты** — реализуют интерфейс `IEffect`.
- **События** — наследуются от абстрактного класса `GameEventBase`.
- **Объекты взаимодействия** — реализуют интерфейс `IInteractable`.

Все компоненты слабо связаны и используют композицию для построения игрового мира.

2 Описание ключевых интерфейсов и абстрактных классов

2.1 Интерфейс ICommand

Отвечает за выполнение пользовательских команд.

```
public interface ICommand
{
    void Execute(Game game, string args);
}
```

Листинг 1: Интерфейс ICommand

2.2 Абстрактный класс CommandBase

Базовый класс для всех команд. Содержит общую логику разбора аргументов.

```
public abstract class CommandBase : ICommand
{
    protected string[] ParseArgs(string args) { ... }
    public abstract void Execute(Game game, string args);
}
```

Листинг 2: CommandBase

2.3 Интерфейс ICondition

Позволяет проверять состояние игры.

```
public interface ICondition
{
    bool IsTrue(IGameState state);
}
```

Листинг 3: ICondition

Где `IGameState` — интерфейс только для чтения состояния.

2.4 Абстрактный класс ConditionBase

Содержит общие поля (например, инвертирование) и метод проверки.

```
public abstract class ConditionBase : ICondition
{
    public abstract bool IsTrue(IGameState state);
}
```

Листинг 4: ConditionBase

2.5 Интерфейс IEffect

Изменяет состояние игры.

```
public interface IEffect
{
    void Apply(GameState state);
}
```

Листинг 5: IEffect

2.6 Абстрактный класс EffectBase

Может содержать общие свойства, например, сообщение для журнала.

```
public abstract class EffectBase : IEffect
{
    public abstract void Apply(GameState state);
}
```

Листинг 6: EffectBase

2.7 Абстрактный класс GameEventBase

Базовый класс для всех событий.

```
public abstract class GameEventBase
{
    protected ICondition condition;
    protected List<IEffect> effects;
    protected bool isOneTime;
    protected bool triggered = false;

```

```
    public void CheckAndApply(GameState state)
    {
        if (isOneTime && triggered) return;
        if (condition.IsTrue(state))
        {
            foreach (var effect in effects)
                effect.Apply(state);
            triggered = true;
        }
    }
}
```

Листинг 7: GameEventBase

2.8 Интерфейс IInteractable

Объекты, с которыми может взаимодействовать игрок.

```
public interface IInteractable
{
    string Id { get; }
    void Interact(Player player, GameState state);
}
```

Листинг 8: IInteractable

3 Конкретные реализации

3.1 Команды

- **LookCommand** — выводит описание текущей локации, объекты и выходы.
- **GoCommand** — перемещает игрока в указанную локацию, проверяя доступность перехода.
- **InteractCommand** — ищет объект по идентификатору в текущей локации и вызывает `Interact`.
- **InventoryCommand** — показывает содержимое инвентаря.
- **HelpCommand** — выводит список доступных команд.
- **StatusCommand** — показывает здоровье и активные флаги.

3.2 Условия

- **HasItemCondition** — проверяет наличие предмета в инвентаре.
- **FlagCondition** — проверяет значение флага.
- **HealthCondition** — сравнивает здоровье с порогом.
- **AndCondition** — логическое И двух условий.
- **OrCondition** — логическое ИЛИ.
- **NotCondition** — инвертирует результат.

3.3 Эффекты

- **AddItemEffect** — добавляет предмет в инвентарь.
- **RemoveItemEffect** — удаляет предмет.
- **SetFlagEffect** — устанавливает флаг в true/false.
- **DamageEffect** — уменьшает здоровье.
- **HealEffect** — увеличивает здоровье.
- **LogEffect** — добавляет запись в журнал.
- **AddExitEffect** — добавляет новый выход из текущей локации.
- **ChangeLocationEffect** — мгновенно перемещает игрока в другую локацию.

3.4 События

- **OnEnterLocationEvent** — срабатывает при входе в локацию.
- **OnTurnEvent** — проверяется каждый ход.
- **OneTimeEvent** — срабатывает только один раз при выполнении условия.

3.5 Объекты взаимодействия

- **Chest** — контейнер с предметами. Может быть закрыт (требуется ключ). При взаимодействии открывается и предметы переходят в инвентарь.
- **Door** — переход в другую локацию. Может быть заперта и открывается при наличии ключа или выполнении условия.
- **NPC** — диалоговый персонаж. Может выдавать задания, реагировать на флаги, давать предметы.
- **Trap** — ловушка. Наносит урон при первом взаимодействии или активируется при входе.

4 Композиция объектов на примере локации

Каждая локация содержит списки объектов, событий и выходов:

```
public class Location
{
    public string Name { get; set; }
    public string Description { get; set; }
    public List<IInteractable> Objects { get; set; }
    public List<GameEventBase> Events { get; set; }
    public Dictionary<string, string> Exits { get; set; } //
                                ->
}
```

Листинг 9: Класс Location

Здесь используется композиция: локация владеет объектами и событиями, но они могут существовать независимо (например, один предмет может быть перемещён в другую локацию). Выходы представлены словарём, что позволяет динамически добавлять или удалять переходы через эффекты.

5 Класс Game и игровой цикл

Класс `Game` содержит текущее состояние, коллекцию всех локаций и индекс текущей локации. Метод `Run` реализует основной цикл:

```
public void Run()
{
    while (!IsGameOver)
    {
        //
        foreach (var loc in locations)
            foreach (var evt in loc.Events)
                evt.CheckAndApply(state);

        //
        foreach (var evt in globalEvents)
            evt.CheckAndApply(state);

        //
        Console.Write("> ");
        string input = Console.ReadLine();
        ProcessCommand(input);

        state.TurnCount++;
    }
}
```

Листинг 10: Игровой цикл

Команды выбираются через словарь `commands` по ключевому слову, аргументы передаются строкой.

6 Соответствие требованиям ТЗ

В таблице 1 обязательные элементы ТЗ и их реализация.

Требование	**Реализация**
Интерфейсы	ICommand, IInteractable, ICondition, IEffect
Абстрактные классы	CommandBase, ConditionBase, EffectBase, GameEventBase
Наследование	Конкретные классы команд, условий, эффектов, событий наследуются от соответствующих базовых классов
Полиморфизм	Вызов методов через интерфейсы (например, применение эффектов через IEffect, проверка условий через ICondition)
Композиция	Location содержит списки объектов, событий; Game содержит GameState и локации
Запрет на проверку типов	Взаимодействие строится на интерфейсах, нет проверок `is` или `as`
Состояние игры в едином месте	Класс GameState
Локации с изменяемой структурой	Выходы хранятся в словаре, эффект AddExitEffect может добавить новый ключ
Объекты с уникальным ID	У каждого IInteractable есть свойство Id
Команды как объекты	Каждая команда — отдельный класс, реализующий ICommand
Условия (простые и составные)	HasItemCondition, FlagCondition, HealthCondition; AndCondition, OrCondition, NotCondition
Эффекты	AddItemEffect, RemoveItemEffect, SetFlagEffect, DamageEffect, HealEffect, LogEffect, AddExitEffect, ChangeLocationEffect
События (при входе, каждый ход, одноразовые)	OnEnterLocationEvent, OnTurnEvent, OneTimeEvent
Квесты (минимум два)	Реализованы через комбинацию условий и эффектов, отслеживаются через флаги (например, "generatorOn")
Минимальный набор локаций	Hall, Storage, DarkCorridor, GeneratorRoom, Exit
Минимальный набор предметов	Key, Torch, Fuse, Wrench, Medkit
Четыре механики из списка	1) Тёмный коридор (урон без Torch); 2) Дверь, открываемая ключом; 3) Генератор, требующий Fuse и Wrench; 4) Одноразовая ловушка

Таблица 1: Соответствие требованиям ТЗ